

Below the Application, Above the Bytes

Joshua Chambers *Spring 2026*

Abstract

Two structural properties of modern computing compound to produce what users experience as platform ownership, meaning the grip platforms have on users and their work, not any technical claim users hold over their data. No layer of the stack stores meaning: data is opaque bytes at every layer below the application, interpretable only by whichever application holds the schema. And the software that interprets that data has migrated off the user's device and onto corporate server farms: applications are services, not programs, and the most capable consumer hardware ever built runs as a set of rendering terminals. Users cannot leave because their data has no meaning without the application, and the application is not something they even run.

This paper argues that both gaps must be addressed at the same level and in the same primitive. It proposes a base layer built on two co-equal pillars: a shared semantic commons and a local-first peer-to-peer substrate. The semantic primitive is the frame, a predicate-role structure drawn from Fillmore's frame semantics and grounded in empirically validated linguistic resources (WordNet, the Collaborative Interlingual Index, FrameNet, VerbNet, ISO 24617-4). Keys refer to meanings, not strings. The substrate is a peer network of local runtimes holding content-addressed, cryptographically signed items built from frames. Frames carry meaning. Items carry identity. Trust drives routing. Computation happens wherever the relevant frames live.

The result is a base layer where data is self-describing and queryable by meaning; where applications are interchangeable local runtimes rather than hosted services; and where the structural asymmetries that define the current platform era dissolve because there is nothing platform-specific left to own. Neither pillar is novel in isolation. Semantic structuring has been pursued for decades; local-first software has an established tradition. What is new, and what this paper attempts, is the claim that both belong in the same primitive, at the same layer, as an open commons.

1. The Semantic Void

In 1945, Vannevar Bush described the central problem of information management: "The summation of human experience is being expanded at a prodigious rate, and the means we use for threading through the consequent maze to the momentarily important item is the same as was used in the days of square-rigged ships" (Bush, 1945). Eighty years later, the maze is incomparably larger, and the threading, while faster, is no less indirect. The reason is structural. It sits below every tool we build on top of it, and it has been there since the foundational layers of modern computing were laid down.

Consider what happens when a user saves a photograph. The filesystem records bytes at a path. The operating system tracks the file's size, its modification time, its location on disk. The image format encodes pixels and, sometimes, a small amount of EXIF metadata: camera model, GPS coordinates, exposure settings. Yet no layer of the stack knows what the photograph is. Who is in it. What occasion it documents. How it relates to other photographs, to the people depicted, to the day it was taken. That knowledge exists only in the user's head, or in the proprietary database of whichever application the user happens to use. The stack has stored the photograph without storing any of what makes the photograph matter.

This is not specific to image files. It is a structural property of every layer beneath the application:

- **The filesystem** sees bytes at paths. Whether those bytes are a novel, a spreadsheet, or a genome sequence is invisible to the layer that stores them.

- **The operating system** sees processes, file descriptors, and memory pages. It cannot distinguish a medical record from a restaurant menu.
- **The network layer** sees packets with source and destination addresses. HTTP adds content-type headers, which identify *format* (text/html, application/json), never *meaning*.
- **The database** sees rows and columns, or documents and collections. Its schema is local to the application that defined it. Two databases storing information about the same person share no vocabulary for describing what they hold.
- **The web** sees pages at URLs. Search engines exist precisely because the web cannot answer “what is this about?” Third parties crawl billions of pages, guess at meaning from word frequency and link structure, and sell access to their proprietary guesses.

Each layer was designed for generality, and each achieves it the same way: treat data as opaque and leave interpretation to the layer above. This is a defensible engineering choice when generality is the goal, but the cumulative consequence is that meaning has no home in the architecture. Every application that needs to do anything with meaning must build its own semantic layer from scratch: its own schema, its own vocabulary, its own query logic, its own integration adapters. The cost is immense but invisible, because it is paid everywhere and attributed nowhere.

The cost takes familiar forms. Every API integration is a bespoke translation between two systems that cannot describe their own contents to each other. Every search engine is a probabilistic compensation for the fact that data does not know what it means. Every data migration is an exercise in reconstructing meaning that was present in the creator’s mind but never captured by the infrastructure. The most recent and most expensive compensation is the large language model. The current wave of AI is, at its core, an effort to recover meaning from data that never stored it. LLMs are trained on vast corpora of human text precisely because meaning lives in the text, not in the infrastructure that holds it. Many of the tasks they perform (classification, extraction, translation, summarization, relationship discovery) are tasks a semantic layer would make trivial, because those tasks reduce to structured lookups when data already carries its meaning. We are building ever-larger statistical models to guess at what could have been recorded directly.

A smaller but revealing instance of the same pattern appears wherever applications exchange key-value pairs, which is nearly everywhere. Configuration files, HTTP headers, database rows, JSON objects, environment variables: the key-value pair is the most fundamental composable pattern in computing. Yet because keys are application-defined strings, they are fractured beyond repair. One system’s `author` is another’s `creator`, another’s `created_by`, another’s `dc:creator`, another’s `writtenBy`. They all mean the same thing, yet no layer of infrastructure knows this. The fragmentation is not cosmetic, it is what makes integration between systems the hardest and most durable cost in the industry.

When no layer below the application stores meaning, meaning must live somewhere, and the only place left is application code. An application’s schema and interpretation logic become the sole route from bytes to whatever users actually care about. Users do not care about bytes. They care about photographs, messages, relationships, purchases, conversations, every one of which is a structured interpretation. Whoever holds the code that produces the interpretation holds the thing users actually value. Data without an application that understands its schema is bytes without value.

Exporting data is therefore possible but insufficient. Standard export mechanisms (GDPR requests, JSON dumps, CSV downloads) deliver bytes accompanied by a schema that is meaningful only within the originating application’s conventions. Another application can interpret those bytes only by reconstructing that schema, a translation that is expensive at scale, lossy at the edges, and bespoke per platform. Users can leave nominally but not practically. The gap between nominal and practical exit is what we experience as platform ownership. It is not a policy that could be legislated away by mandating more export formats, it is the shape of the substrate. Every data-portability regulation confronts the same underlying constraint: bytes move, meaning does not.

A semantic base layer would alter this equation at its foundation. If data carries its own meaning at the moment of creation, any application that understands the shared vocabulary can interpret

it, so exit cost approaches zero and the application becomes interchangeable. The bundle of data, schema, interface, social graph, and moderation that platforms sell as one thing unbundles, because each component becomes expressible in the same substrate. What we call the platform's moat, the structural advantage that makes leaving impractical, does not survive that unbundling; it dissolves not through regulation or competition but because there is nothing bundle-specific left to own.

That is the first of two structural gaps that together produce the platform era. The other operates at a different layer of the stack, and its emergence is more recent. The next section turns to where the software that interprets your data has gone.

2. The Age of SaaS

Consumer computing has passed through two completed eras and is deep into a third. The trajectory has not been linear. The current moment is, in a specific sense, a return to something the industry already left behind, with the critical difference that the centralization is now a commercial choice rather than a hardware necessity.

In the mainframe era of the 1950s through the 1970s, compute was centralized because it had to be. The hardware was expensive, rare, and physically immobile, and users reached it through terminals with just enough intelligence to negotiate a session. All real work happened in the machine room. This was not consumer computing in any meaningful sense; mainframes were institutional. Still, the arrangement established a pattern: compute in one place, access from another, the asymmetry between them non-negotiable.

The personal computing era of the early 1980s through the mid-1990s was the first time ordinary people could run real software on their own machines. The IBM PC, the Macintosh, and their peers put meaningful compute on individual desks. Software came on floppies and later on CDs, installed locally, and ran locally. The user's data lived on the user's disk. The software itself was almost always proprietary, closed-source, and licensed rather than owned, but the locus of computation and state was local. The user ran the software, held the data, and controlled when the software ran, and when it stopped. The vendor controlled the code; the user controlled the operation.

The networked era began in the mid-1990s and continues in steadily deepening form. The web started as a document-retrieval system and evolved into an application-delivery system through incremental accretion: HTML learned forms; JavaScript learned to manipulate the page; AJAX learned to talk back to servers without reloading. The browser slowly became a runtime, but the runtime's job was to render what a server decided, and each accretion moved the locus of computation further from the user's machine.

What began as hypertext documents has become software as a service (SaaS). Applications are no longer something you run, they are something you access. They live in data centers you do not operate, on infrastructure you do not own, under subscriptions you do not escape without losing access to your own work. Nearly every productive tool a modern user touches (email, documents, design work, communication, project management, calendar, photos, notes, code hosting, source-control hosting, payment processing) is a service running on someone else's computers. Even the tools that retain a local presence, like Slack or Discord or Photoshop or 1Password, are clients for remote services, useless if the service goes away.

The web protocol carries part of the responsibility for where this has led. HTTP was designed for document retrieval: a client asks a server for a thing, the server sends it. Everything interactive we have built on that foundation has preserved the direction of authority. The server holds state, runs logic, and decides what to send; the client renders. JavaScript in the browser has closed some of this gap, but the direction is unchanged: the client's code is code the server chose to ship, running against data the server chose to expose. When the server goes away, the client becomes decoration.

The most powerful consumer computers ever built are being used as near-dumb terminals. The phone in a user's pocket in 2026 has more processing, more memory, more storage, and more bandwidth than the mainframes of the 1970s. A modern laptop has more capacity than most users' everyday workloads come close to using. Yet very few consumer-facing applications actually run on these machines; the device renders what a server farm computes. The sophistication of the hardware is spent on rendering engines rather than on the work the user came to do. This is not a technical outcome but a business outcome, imposed on users whose hardware could do vastly more if anyone would let it.

Running the computation is how you own the decisions, and the set of decisions a server makes on a user's behalf is larger than users typically consider. Recommendation order, feed curation, content moderation, search ranking, fraud detection, A/B test assignment, price discrimination based on account signals, the shape of what is shown and the timing of when it is shown, the filtering of what is hidden. All of this happens on servers, inside code users cannot inspect, on data users cannot access, subject to policies users did not agree to and cannot amend. Even if a user's data were semantic and portable, the decisions made about that data would still belong to whoever was running the application logic. Platforms own not just the meaning of data but the agency over it.

Data opacity and the SaaS era are complementary structural causes of platform ownership, and fixing only one is insufficient. Semantic data trapped on server-side infrastructure is no more portable than opaque data on server-side infrastructure. Local compute operating on opaque data is no more powerful than remote compute operating on opaque data. Together the two produce the condition users experience in the current era: your data means nothing without the application, and the application is not something you run.

What is missing is not a better search engine or a smarter parser, and not a different platform hosting the same structural pattern. What is missing is a layer where meaning is the fundamental unit of storage, identity, and retrieval, and where that layer lives on hardware the user actually controls. Both gaps must be closed in the same layer, or neither closure is effective.

3. How We Got Here

Neither gap is the result of inattention. The historical conditions that produced them have been understood for decades, and serious attempts to close them have been made on both sides. Each deserves a closer look, because the failure patterns explain why the substrate itself, rather than another addition to it, is the only way forward.

When the foundational layers were laid down in the 1970s, nodes were disconnected and bytes were precious. The byte-stream abstraction (everything is a file, a file is a sequence of bytes) was a practical triumph given the constraints. TCP/IP, HTTP, SQL: each subsequent layer solved the problem in front of it with the resources available. A semantic data model was not rejected, it was beyond the horizon. The centralizing trajectory of the commercial web was even further out.

The semantic gap persisted in part because the linguistic foundations for closing it took decades to mature. A semantic key cannot be a string; it must refer to a stable, language-independent concept with a hierarchy, cross-lingual equivalents, and participation in structured scenes. Building those objects requires empirical research into how meaning is structured across human languages. The resources that make it tractable (WordNet, CILI, FrameNet, VerbNet, ISO 24617-4) are products of computational linguistics that have only recently reached the maturity needed to serve as a practical foundation. And once they did, the commercial landscape of the 1990s and 2000s ran in the wrong direction: every major platform held its data models close because controlling the model meant controlling the ecosystem. Interoperability was a competitive threat to the kind of cross-organizational collaboration a shared semantic foundation requires.

The locality gap has a different shape. The hardware became capable enough for serious local

computation by the mid-1990s and has only grown more so. What drove computation away from users was not a technical limit but a convergence of commercial incentives: cloud hosting got cheap, network effects rewarded centralization, and the subscription business model worked perfectly for services and not for shipped software. For any product taking shape in the 2000s and 2010s, a hosted service was easier to monetize, easier to update, easier to monitor, and easier to prevent users from leaving. A generation of developers grew up with SaaS as the default mental model rather than as the historical anomaly it is. Local-first and peer-to-peer architectures remained technically viable throughout this period but lacked the commercial pull, carried as a counter-current by specific communities (Kleppmann’s academic work, Secure Scuttlebutt, IPFS, and others) without displacing SaaS as the industry default.

That ambient state has been the backdrop for a second history: deliberate attempts to retrofit semantic structure onto opaque substrates and decentralization onto server-centric ones. None has become foundational. Their failures, taken together, point at the same diagnosis.

The semantic retrofits

The Semantic Web is the most ambitious attempt on the semantic side. Berners-Lee’s 2001 vision described a web in which “information is given well-defined meaning, better enabling computers and people to work in cooperation” (Berners-Lee et al., 2001). The technical realization (RDF triples, OWL ontologies, SPARQL queries) is rigorous and powerful. Twenty-five years later, RDF is widely used in specialized domains (biomedical ontologies, library science, government data) but has not become a general-purpose semantic layer. The web remains overwhelmingly opaque bytes at URLs.

RDF’s genuine strengths are substantial: a universal graph model, formal inference via RDFS and OWL entailment, a powerful query language in SPARQL. In specialized domains where those capabilities matter, RDF has proven its value. Three structural problems kept it from becoming general-purpose.

First, RDF annotates existing resources. It is layered *on top of* the web, not *built into* it. A web page can exist without any RDF. Most do. The semantic annotation is optional, which means it is absent in the vast majority of cases. The cost of creating semantic metadata falls on the producer while the benefit accrues to the consumer: a classic misaligned-incentive problem.

Second, RDF requires the author to commit to an ontology (Gruber, 1993). In practice, choosing and using an ontology correctly is hard. It requires expertise that most content creators do not have and are not motivated to acquire. The Semantic Web effectively asks every web author to be a knowledge engineer.

Third, the annotation is disconnected from the content. The RDF description of a web page is a separate artifact from the page itself. It can become stale, incorrect, or inconsistent without any mechanism to detect the divergence.

Concrete systems built on the full RDF/OWL/SPARQL stack, such as the Open Semantic Framework (Structured Dynamics, 2009), confirmed these limitations in practice: technically rigorous, adopted in narrow domains, but unable to achieve the general-purpose uptake the vision required. The project went quiet after 2016.

Schema.org addressed some of these problems by providing a single vocabulary backed by major search engines. Its adoption is broader than RDF/OWL, precisely because it is simpler and because search engines provide a direct incentive (better rankings) for using it. Schema.org remains a metadata annotation regardless: a sprinkle of JSON-LD in an HTML header. It describes pages *about* things, not the things themselves.

Dublin Core, **EXIF**, **ID3 tags**, **OpenGraph**, and dozens of other metadata standards each solve a narrow problem. They do not compose. A photograph with EXIF data and a document with Dublin Core metadata cannot be queried together because they share no vocabulary, no addressing scheme, and no common notion of what “subject” or “creator” means.

The structural lesson from these efforts is crisp: **you cannot make a semantically inert layer semantic by annotating it**. The annotation is always optional, always disconnected from the content, always maintained by a different process, and always expressed in a vocabulary local to one standard or domain. The layer itself remains opaque.

The locality retrofits

A parallel set of attempts has tried to retrofit decentralization and user control onto an infrastructure whose business model depends on their absence. Each of these efforts was built by thoughtful practitioners and has adoption within specific communities. None has become the default.

XMPP (originally Jabber, 1999 onward) was an earlier federation attempt focused on messaging and presence. It saw significant adoption in the 2000s, including as the transport for Google Talk and early Facebook Chat. That adoption proved instructive: once the largest deployments sat in proprietary hands, those operators withdrew, leaving the ecosystem without the network effects that had made federation useful. A cautionary tale about relying on large commercial adopters to carry a federated protocol.

The Fediverse (Mastodon, Pleroma, and other ActivityPub-based systems) provides a federated alternative to centralized social media. Users choose an instance, and their accounts interact across instances via ActivityPub. The model genuinely decentralizes operation: there is no single company whose servers must run for the network to function. Its adoption grew substantially after high-profile disruptions at centralized platforms. ActivityPub is a federation protocol, however, not a local-first substrate. User data lives on the chosen instance's servers. Moving between instances is an operation that often loses history or followers. The instance operator remains a gatekeeper for the user's experience; this approach has distributed the gatekeepers, not removed them.

Solid (Berners-Lee and collaborators, 2016 onward) proposed personal data “pods” : user-controlled storage that applications request access to. The direction is correct, but Solid layers atop HTTP and OAuth, and applications still carry proprietary schemas for what the pods hold. Without a semantic substrate underneath, the pod is another storage location rather than a reorientation of where meaning lives.

Secure Scuttlebutt (Tarr et al., 2019) demonstrated a fully peer-to-peer social protocol with cryptographic identity and append-only signed logs. Technically it is closer to what a local-first substrate should do. Adoption required users to manage their own identities and accept a user experience shaped by a research-grade protocol, however. The community that embraced it has been small and committed, and SSB has not displaced mainstream social software.

FreeNet (Clarke et al., 2001) was one of the earliest sustained attempts at a peer-to-peer substrate for publishing and retrieval. It introduced content-addressed storage routed through a distributed overlay, on the premise that data could be held by peers without any central server knowing where it lived. Its design directly influenced much of what followed, and its continued development across more than two decades, including a substantial recent rewrite, demonstrates both that the technical approach is viable and that it still rewards fresh thinking. FreeNet has not reached the mainstream because its user experience, content model, and threat-model trade-offs shaped it for a specific community, anonymity-focused publishing, rather than for general-purpose use.

BitTorrent (Cohen, 2003) demonstrated at massive scale that peer-to-peer distribution works when the architecture fits the problem. In its trackerless form, using a Kademlia DHT, it operates without any central coordinator at all, and it has carried significant fractions of global internet traffic for over two decades. It solves bulk file transfer extremely well, but does not attempt to be a substrate: files remain opaque, with no common data model, identity, or semantic structure.

Git (2005) is the most widely used distributed system in the world and an instructive case: technical decentralization can survive while cultural centralization takes hold anyway. Each clone is a full repository; content-addressing via SHA-1 (migrating to SHA-256) making every object identifiable

by its content; any repository can sync with any other over any transport; no single server is architecturally required. Yet the surrounding workflow (pull requests, issue tracking, CI, code search, discovery) became the value proposition of GitHub and a handful of similar platforms (GitLab, BitBucket, and others), and the developer community centralized around them despite Git itself being fully distributed. The lesson cuts against complacency about technical decentralization: it is necessary but not sufficient. If the workflow and social layers around the artifact become the real center of gravity, those layers become the point of centralization, and the underlying tool's distributedness does not save users from a new gatekeeper.

IPFS (Benet, 2014) builds on FreeNet's lineage, on BitTorrent's chunked-distribution model, and on the broader distributed-hash-table research tradition (Chord, Kademlia, Pastry, Tapestry and their descendants) that made decentralized lookup practical at scale. It provides content-addressed storage and a peer-to-peer distribution network, solving the real problem of moving bytes between peers without a coordinating server. IPFS on its own, however, is a storage layer. The data moving through it is still opaque. Without a semantic substrate, a file retrieved from IPFS is the same bytes one would retrieve from any CDN, with the same interpretation problem.

The AT Protocol (underlying Bluesky) and **Matrix** take different approaches to federation with real trade-offs around identity portability and decentralization. Neither carries data with meaning in the sense this paper means; both remain federated services rather than a reoriented substrate.

Two structural lessons emerge, mirroring but distinct from the semantic one.

You cannot make a server-centric layer local-first by federating it. Federation (Fediverse, Solid, AT Protocol, Matrix) distributes the servers but preserves the client-server boundary. The data still lives on servers; interpretation still requires application code that lives on servers; agency over what happens to the data still belongs to whoever operates the server. The problem was never the number of servers; it was the architectural privilege of being a server.

Peer-to-peer transport without meaning is not a foundation. The fully peer-to-peer attempts (FreeNet, BitTorrent, SSB, IPFS) solve transport and storage, each in its own way, but each carries opaque bytes rather than meaningful frames. A user who wants to do anything with the data still needs application code that understands it, and that code still lives wherever its publisher chose to host it. Peer-to-peer without meaning is a sophisticated file-transfer mechanism, not a substrate on which applications become interchangeable.

The common lesson

The three patterns of retrofit converge on a single diagnosis: additions cannot compensate for a substrate whose shape is wrong for the job. Annotation does not make opaque layers semantic. Federation does not remove the client-server boundary. Peer-to-peer transport does not carry meaning. Each is an addition to an architecture that was not built for what the addition requires, and each fails in the specific way the underlying architecture leaves no room for it to succeed. Closing either gap requires a different substrate, not another addition to this one.

The solution must be a *layer* where creating data is simultaneously creating semantic structure and locating that data in a user-controlled peer. The two properties are not separate operations, and neither can be supplied by annotation, federation, or transport alone.

What's different now

Four things have converged to make such a substrate possible now in a way it was not before. First, the computational linguistics infrastructure matured: WordNet, CILI, FrameNet, VerbNet, and ISO 24617-4 collectively provide the grounded vocabulary the semantic pillar needs. Second, the technical infrastructure for distributed state matured as well: CRDTs have become practical, signing cryptography is cheap enough to apply at the per-message level, content-addressing is ubiquitous (every Git commit is a use of it), and local-first synchronization has moved from research topic to

shipping practice. Third, global interconnection made shared vocabularies both necessary and viable; the network that makes the semantic problem acute is the same network that makes a collaborative solution practical, and the same network over which a peer substrate would operate. Fourth, the open-source movement created the collaborative environment such a commons requires. The linguistic databases carry permissive licenses. The cryptographic foundations are open. The storage and networking building blocks are open. A base layer along both pillars is inherently a commons: it only works if shared, and it can only be shared if open. That commons is now possible in a way it was not during the era of proprietary platform wars and server-centric default architectures.

What such a base layer would actually require is the subject of the next section.

4. What a Base Layer Requires

If neither a semantic layer nor a local-first substrate can be achieved by annotating, federating, or moving bytes peer-to-peer on top of what we already have, what must a new foundation look like? The missing layer sits above the raw byte substrate that filesystems and networks already provide, and below the application that currently monopolizes interpretation; hence the title. The answer to what it requires has two halves. The first concerns meaning: what must data be for any application to interpret it? The second concerns locality: where must the data and its interpretation live?

(A note on terminology: “semantic layer” is already used in the data analytics industry to mean a translation layer between database schemas and business concepts, as in tools like Looker and dbt. What is meant here is different. A BI semantic layer sits above the data and translates queries; the layer proposed here would sit *with* the data, because the data would already know what it means.)

Grounded predicates, not strings

A semantic layer requires keys that carry *meaning*, not just labels. The key must refer to a concept, not a string, and that concept must be shared across systems, applications, and languages.

The problem of vocabulary sharing across systems was formalized in foundational ontology research. Tom Gruber (1993) argued that systems cannot meaningfully share knowledge without committing to shared vocabularies whose terms have agreed-upon meanings, and laid out design principles (clarity, coherence, minimal ontological commitment, and others) for the ontologies such sharing requires. The Semantic Web pursued this insight through URI-identified predicates. URIs, however, are locations, not meanings. They are globally unique, but they do not carry semantic content intrinsically. Two different URIs can denote the same concept (`schema.org/author` vs. Dublin Core’s `dc:creator`), and nothing in the infrastructure connects them.

What we need are keys that refer to *meanings*: language-independent, application-independent units of semantic content with stable identities. Computational linguistics provides exactly such units, developed in detail in sections 5 and 7. For now, the requirement: keys must be grounded meanings, not strings.

Structured assertions

A flat key-value pair (`author: Tolkien`) captures a single relationship but loses the structure that gives it meaning. Who is asserting this? About what? In what capacity? A key-value pair has no structure to express the *kind* of relationship, the *participants* and their roles, or the *context* in which the assertion holds.

What we need is the frame pattern: a **predicate** that defines a structured assertion, and **role bindings** that fill its slots with values. The theoretical foundation for this pattern comes from Fillmore’s frame semantics (1968; 1982), and the empirical grounding comes from decades of computational-linguistics

research (FrameNet, VerbNet, ISO 24617-4) that has catalogued and standardized the ~25 universal thematic roles human languages use to describe who did what to whom, where, when, how, and why (Youn et al., 2016). The next section develops the primitive in detail; for now, the requirement is the shape: predicates and role bindings, not flat key-value pairs.

Write-time resolution

This is the core inversion. Every existing system stores data first and tries to determine its meaning later. Search engines crawl, NLP systems annotate after the fact, data integration pipelines map between schemas post-hoc. All of these are attempts to recover meaning that was present in the creator's mind but never captured in the data.

A semantic base layer inverts this. Meaning could be resolved *at the moment of creation*, when it is trivially easy, because the creator knows what they mean. The disambiguation that search engines and NLP pipelines struggle to perform after the fact is effortless at write time. When a user creates a relationship between a person and a book, they know whether they mean “authored,” “edited,” “reviewed,” or “purchased.” If the layer captures that distinction as a grounded semantic predicate at creation time, no subsequent system ever needs to guess.

The predicate, once chosen, tells the system what roles to expect. The system can prompt for them, offer completions, validate inputs. The act of creating data *becomes* the act of resolving meaning, because selecting a predicate and filling its roles is inherently a semantic operation.

This is not natural language understanding. Such a layer need not parse free text and try to extract meaning. It would structure the input environment so that meaning is captured as a natural consequence of creation. The user selects a predicate, fills roles, and the result is a grounded semantic structure. The hardest problem in NLP (disambiguation) is trivially solved at write time by the person who knows what they mean.

Cross-lingual stability

A semantic layer that works only in English is an English-language metadata standard, not a semantic layer. The concept that English speakers call “dog,” Spanish speakers call “perro,” and Japanese speakers call “犬” is the same concept. A semantic layer must represent meanings independently of the words that express them.

This requires a clean separation between *meanings* and *words*. Meanings (which I will call **sememes**, following usage in structural semantics) are language-neutral units with stable identities. Words, in their role as language-specific expressions that point to meanings, are called **lexemes**. Each lexeme belongs to a particular language and carries the morphological apparatus of that language (inflection, conjugation, case, gender, tense). Multiple lexemes across languages can point at the same sememe: the English “authored,” the Spanish “escrito,” and the German “verfasst” are three lexemes, one sememe. The predicate AUTHORED exists independently of any of them.

The four requirements above describe data. Four more describe where the data lives and who runs the code that interprets it.

Cryptographic identity

Identity would need to be a keypair rather than a row in a registry. A user should not need to register with a service or keep an account active on a server to exist as a participant. Identity can be self-sovereign, verifiable anywhere, and independent of any specific operator. This is the minimum condition for agency without a gatekeeper, and the foundation on which signatures, trust relationships, and content attribution would all rest: each of those builds on identity being a thing users hold rather than a thing services grant.

Content-addressed data

Data would be identified by what it is, a hash of its content, rather than by where it lives. The same data retrieved from any source would then be recognizably the same data, and its integrity verifiable locally. Content-addressing (Merkle, 1979; Benet, 2014) is what decouples identity from storage location, which is in turn what makes data portable between peers without losing its meaning, provenance, or relationship to other data. Storage would become commodity; hosting would become a user-revocable choice rather than a platform commitment.

“Social networking”

Identity is a keypair and data is content-addressed, yet neither on its own answers a practical question: how does a specific piece of data reach a specific user? Something has to decide where data goes. In a centralized architecture, the server knows, because it holds both the data and the user’s connection to it. A substrate without servers cannot use that answer.

Peer-to-peer systems have historically routed by arbitrary distance metrics over a hash space (Freenet, Kademlia, Chord, Pastry and their descendants). A peer is “close to” a content ID not because it cares about the content but because of a mathematical property of the ID. This works technically but treats all peers as interchangeable and all content as equally relevant, which is rarely true in practice.

Data is not uniformly relevant. Every person cares about specific data: their own work, data from people they know, data about topics or places or communities they follow. Every organization is the same, caring about its own operations and about data from specific partners, customers, and peers. Relevance follows relationships, between individuals, between organizations, between communities, between devices and the people who use them.

A routing mechanism that respects this structure would carry data along the relationships that make it relevant. Sociologists have studied such structures under the name *social networks* for more than half a century (Barnes, 1954 and the social-network-analysis tradition that followed), as the shape through which information, influence, and resources actually flow among humans. The commercial platforms that adopted the phrase “social networking” in the late 1990s and 2000s built closed enclosures around a small slice of this phenomenon and sold access to it. The substrate proposed here is aimed at the underlying structure rather than any particular enclosure of it: a network whose topology is the social graph itself, not a product that owns and resells it.

The claim that social structure suffices for routing is not just philosophical. Human social networks empirically exhibit *small-world* properties (Milgram, 1967; Watts & Strogatz, 1998): despite their sparseness, most pairs of people are connected through a small number of intermediate relationships. Data routed along the social graph can therefore reach any particular user through a handful of hops on average, even at scale. The efficiency is not an algorithmic property of the substrate but an empirical property of the network the substrate traces.

Users of today’s centralized services route through relationships inherited rather than chosen: the employer’s cloud, the default search engine, the platform where the relevant people already happen to be. Replacing inherited structure with deliberate choice, expressed in the substrate itself, is the shift. What individual users and applications decide about those relationships - which peers to accept data from, whose relays to use, whose endorsements to credit - is a trust question. Trust is large enough to be its own subject, and the substrate surfaces it without pretending to settle it. What the substrate provides is the primitive underneath: signed, content-addressed assertions carried along deliberately-chosen relationships.

The arrangement serves as both a routing mechanism (data reaches the peers who actually care about it) and a privacy mechanism (data flows only along peers deliberately included in the path).

Local execution

The runtime that interprets data and acts on it would run on the user's machine. A protocol describes how bytes move; a substrate is also the thing that runs on both ends to turn those bytes into meaningful action. The user's device would be a full participant rather than a renderer, and the computation that turns data into experience would happen there.

These requirements do not name a structure. They constrain one. Whatever fits has to be built around meaning rather than strings, carry role-keyed values rather than flat attributes, be complete at the moment of writing, survive translation between languages, be identified by content rather than location, be signed rather than anonymous, route along deliberately-chosen relationships rather than through central brokers, and execute locally rather than remotely. None of these requirements is individually novel. What would be new is asking a single structure to satisfy all of them at once, and to do so as the foundation of a layer rather than an annotation or federation laid over one.

5. The Frame as Primitive

Earlier, we arrived at the frame as a shape that satisfies the semantic requirements. Fillmore supplied the form for a different purpose: analyzing what sentences mean rather than structuring data. Using it as a data primitive is the move the linguistic literature never had occasion to make. The four locality requirements attach to the primitive and to what holds collections of frames together, which the next sections develop.

A semantic frame, in this usage, is:

```
Frame {  
    predicate:  a grounded meaning      (what kind of assertion)  
    bindings:   role-value pairs        (the semantic content)  
}
```

A predicate and its role bindings. Nothing else is structurally required. Every element of the frame (what it asserts, what it's about, who is involved, what content it carries) is expressed as a role binding on the predicate.

A **title assertion**: predicate TITLE, bindings (THEME) = the-book, (VALUE) = "The Hobbit". The predicate TITLE defines two roles: what is being titled, and what the title is.

A **chess move**: predicate MOVE, bindings (LOCATION) = the-game, (AGENT) = Fischer, (THEME) = king-pawn, (SOURCE) = e2, (GOAL) = e4. Location (which game), Agent (who moved), Theme (what piece), Source (from where), Goal (to where). A single move is a single semantic assertion.

A **video**: predicate VIDEO, bindings (THEME) = the-movie, (VALUE, MKV, UHD) = master-file, (VALUE, MKV, HD) = transcode. The same VALUE role carries content in different formats, distinguished by the qualifiers that follow it in the compound key.

An **authorship assertion**: predicate AUTHORED, bindings (THEME) = The Hobbit, (AGENT) = Tolkien.

These are all structurally identical: a predicate and role bindings. The predicate determines what roles the frame expects; the roles determine what the values mean.

The predicate is worth pausing on, because it is easy to treat it as a distinct kind of thing. It is not. It is a sememe, a unit of meaning from the shared vocabulary, acting in a particular structural role. In that role, a sememe serves as a template for the frame: it declares what bindings a frame of this kind is expected to carry and how those bindings relate. Calling a sememe a *predicate* names the role it plays, not a category it belongs to. The same vocabulary must supply everything. TOLKIEN, HOBBIT, AUTHORED, and TITLE are all meanings. Which of them naturally fits the predicate role is a matter of what each one denotes, not a structural constraint. Meanings that name relations or

events (AUTHORED, TITLE, MOVE) naturally fit as predicates. Meanings that name kinds of things or instances thereof (HOBBIT, TOLKIEN, CHESS) naturally fit as binding values or, as we will see, as templates for items.

Two levels of role

The ~25 universal thematic roles (Agent, Theme, Goal) are powerful because they are universal, but they are also general. In a chess game, both Fischer and Spassky are Agents. Calling them both “Agent” is correct but insufficient. We would need to say they are *players*, and that one plays white and the other black.

FrameNet and VerbNet resolve this tension differently. FrameNet defines frame-specific elements: the Commerce_buy frame has Buyer, Seller, Goods, Money. VerbNet maps these back to universal roles: Buyer maps to Agent, Goods maps to Theme. Both levels are useful. The universal level enables cross-frame queries (“all frames where someone is an Agent”). The frame-specific level enables precision (“all frames where someone is a Buyer”).

A layer built this way would need both levels, connected through the vocabulary’ s inheritance hierarchy. PLAYER, BUYER, and AUTHOR are all meanings in the shared vocabulary, each a *specialization* of the universal role AGENT. The relationship can be expressed in the vocabulary’ s own terms: HYPERNYM { (THEME) = PLAYER, (GOAL) = AGENT }. PLAYER is a kind of AGENT. The vocabulary describes itself with the same primitives it uses to describe everything else.

A PLAYER frame on a chess game would use the PLAYER role, not the generic Agent, because PLAYER carries the additional meaning the context needs. Because PLAYER inherits from AGENT, however, any query at the universal level would still work: “all frames where Fischer is an AGENT” would find chess games, authorship assertions, and anything else where Fischer acts intentionally.

Compound keys

Roles can be further qualified through **compound keys**: sequences of meanings that together identify what a binding is. A movie might have (VIDEO, MKV, UHD) and (VIDEO, MKV, HD). A document might have (GLOSS, ENGLISH) and (GLOSS, SPANISH). The compound key is a sequence of meanings (role plus qualifiers) that together specify the binding with arbitrary precision.

Every element of a compound key is a grounded meaning. VIDEO is not a MIME type prefix; it is the meaning “moving visual content.” MKV is not a file extension; it is the meaning “Matroska multimedia container format.” ENGLISH is not a locale string; it is the sememe for the English language.

And every meaning in a compound key is an *opportunity for indexing*. If a layer built this way indexes frames by the meanings in their binding keys, then “show me all videos” becomes a simple index lookup on VIDEO. “Show me all UHD videos” narrows to frames whose keys include both VIDEO and UHD. “All MKV content” finds every frame with MKV in its key. The compound key would function as a multi-dimensional index built from the vocabulary itself. No separate tagging system, no search facets, no metadata catalog. The key is the index.

Everything is a role binding

There is no fundamental distinction between “the data” and “the metadata” of a frame. A title’ s text, a video’ s master file, a chess move’ s destination square, a document’ s author: each is a role binding. Provenance is a binding. Signatures are bindings. Timestamps are bindings. What we call “data” is a value filling a role. What we call “metadata” is also a value filling a role. The distinction is conventional, not structural.

Beyond the natural-language inventory

The ~25 thematic roles inherited from linguistics describe the participants in events: who did what to whom, where, when, how, why. For a frame primitive that has to stand in for everything a data layer stores, they are necessary but not quite sufficient. A few additional roles emerge as soon as the frame is asked to do work that natural language did not need to do.

The first gap is already visible in the title example. The actual text “The Hobbit” is not an Agent, a Theme, or a Goal; it is content, the value the predicate asserts about its theme. **VALUE** generalizes VerbNet’s narrow Value role into the role for whatever a predicate carries as its payload: a name, a quantity, a measurement, a piece of text, a binary blob.

The second gap is operational. How should an assertion be handled once made? Replicated? Encrypted? Retained for how long? **CONFIG** is the role for policy on the assertion itself: (CONFIG, REPLICATION), (CONFIG, PRESENTATION), (CONFIG, RETENTION), and so on. Any frame can carry CONFIG bindings regardless of its predicate.

The third gap is causal ordering. A chess move follows the previous move; a paragraph edit follows the edit before it; a reply follows the message it answers. **FOLLOWS** is the role for causal or temporal predecessors, and like CONFIG it is cross-cutting: any frame can carry a FOLLOWS binding pointing at an earlier frame.

Three roles added to the linguistic inheritance: VALUE (generalized), CONFIG (new), FOLLOWS (new). Each names a function the linguistic literature had no need to catalog.

Frames as portable units

A frame carries within itself everything needed to interpret it: its predicate names the kind of assertion, its binding keys name what each value means, and the values carry the content. No external schema is required, no application-specific decoder ring, no lookup against a central authority. A frame retrieved from any source is legible on arrival to any runtime that understands the shared vocabulary. That self-sufficiency is what makes the frame a unit of transit between peers as well as a unit of storage. Signed, it is a verifiable assertion that can travel through any route and arrive intact. Content-addressed by the hash of its predicate and bindings, it is identifiable as the same frame regardless of where it was retrieved from. These properties are not an addition to the frame primitive; they are consequences of what the frame already is.

Predicates carry behavior

So far we have treated a predicate as a structural template, declaring what bindings a frame expects. A predicate can do more. It can also declare how frames of its kind behave: how they might be expressed in text or other input, how they might be evaluated if they carry a computation. These declarations would be data on the predicate itself, not rules maintained by a separate parser or interpreter.

Consider the token +. In ordinary treatment, + is a symbol that a language’s grammar rules know how to parse. In a frame-based layer, the picture is different. + is not a meaning. It is a *token*, a written symbol used in some notations. Other notations use different tokens for the same idea: “plus,” “más,” “加える.” All of them point to the same underlying meaning, the sememe ADD. That sememe, in its role as a predicate, can declare the properties a parser would need to know about it: it is infix, it has a precedence, it associates left-to-right. These properties would not be grammar rules the parser has to be told in advance. They would be data the parser reads off the predicate when it encounters one of its tokens. No separate precedence table. No grammar.

This extends to structural symbols. Parentheses are tokens whose corresponding meanings declare “I open a group” and “I close a group.” There is no reserved syntax. Everything (verbs, operators,

functions, parentheses, commas) would resolve through the shared vocabulary. Syntax becomes vocabulary.

Any domain can bring its own notation. Chess algebraic notation, regular expressions, mathematical symbols: each is a set of tokens whose corresponding predicates declare how they parse, resolved through the same mechanism as arithmetic operators or English prepositions.

The behavior a predicate declares is best understood as a *contract*, not a piece of code. The contract lives with the predicate in the vocabulary; code that satisfies the contract (an actual parser, an actual evaluator) is something else entirely. Where such code comes from and how it gets attached to a predicate is a question the primitive itself does not answer. That question is taken up later.

6. What Frames Cohere Around

Frames are the primitive, yet a single frame is rarely the whole story. A book is a TITLE frame (“The Hobbit”), an AUTHORED frame (Tolkien), TEXT frames (the chapters), a COVER_ART frame, a PUBLICATION frame (1937, Allen & Unwin), and more. Each frame is a separate assertion with its own predicate and bindings, and they are all *about the same thing*. They only make sense together.

If frames can be about the same thing, they need a shared identity to point to. That identity, and the collection of frames cohering around it, is what we will call an **item**.

An item is not a new primitive in the way a frame is. It is what falls out when frames need to be *about* something: a stable identity that frames can reference to indicate “I am about *this thing*.” The book is an item. Tolkien is an item. A chess game is an item. Each exists as an identity around which frames accumulate, building up a coherent, multi-faceted description. The role that carries this reference depends on the predicate: THEME for an authorship assertion (the work being described), LOCATION for a chess move (the game where it happens).

Identity in this picture uses content-addressed cryptography: an item’ s identity is a hash derived from its defining characteristics, making it stable, verifiable, and independent of any central registry. Identity is not assigned by an authority but established by the convergence of content. An item can live on any device, or on many, and still be recognized as the same item; the book, the chess game, the photograph are not located in any particular database but identifiable wherever they happen to be stored. The choice has consequences, and those consequences are load-bearing parts of what follows.

Items change over time: a book gains a new edition, a chess game accumulates moves, a photograph acquires reactions. Each change produces a new version, identified by the hash of the item’ s current manifest (the signed list of frames it endorses at that point). The item’ s identity (IID) is stable across versions; each version (VID) is a snapshot. Version history is a chain of manifests, each pointing at its predecessor. Peers holding the same item may hold different versions, and synchronization is a matter of exchanging the manifests and frames each is missing. Conflict resolution when two peers have diverged is an area of active design, informed by CRDT research and by the practical experience of systems like Git that manage divergent histories.

The archetype

What makes a particular item the kind of thing it is? A book is recognizably a book, not because some authority declares it so, but because the frames it carries are the frames a book is *expected* to have. There are TITLE frames. There are AUTHORED frames. There are TEXT frames for the chapters. There are PUBLICATION frames. The collection of expected frames is what makes the item, in any meaningful sense, a book.

Where does that expectation live? In the same place predicates live: the shared vocabulary. BOOK is a sememe, just like AUTHORED is a sememe. Where AUTHORED, in its role as a predicate, declares

what bindings a frame of its kind expects, BOOK, in a different structural role, declares what frames an item of its kind is expected to carry. Call a sememe playing this latter role an *archetype*. BOOK is an archetype. CHESS_GAME is an archetype. PERSON is an archetype. LANGUAGE is an archetype. Each is a sememe in the shared vocabulary, acting as a template for items rather than for frames.

Predicate and archetype are not categories of sememe. They are functional roles a sememe can play, parallel and complementary. A predicate is a sememe acting as a template for a frame. An archetype is a sememe acting as a template for an item. The same vocabulary supplies both. Which role a particular sememe naturally fits depends on what it denotes: meanings that name relations or events fit as predicates, meanings that name kinds of thing fit as archetypes. Nothing in the system would enforce the assignment, but in practice the division falls out cleanly from the meanings themselves.

There is one important asymmetry between archetypes and predicates, and it shapes how items grow. A predicate's declaration is mostly closed: a frame using a particular predicate carries the bindings the predicate calls for, though cross-cutting structural roles like CONFIG can appear on any frame regardless of predicate. An archetype's declaration is *open*: it lists the frames an instance is expected to carry, but instances can accumulate frames the archetype never mentioned. A book is expected to have TITLE and AUTHORED frames. Nothing prevents anyone from also attaching a LIKE frame, a comment, a citation, a fact-check, a translation, a review, a bookmark, a personal annotation. The archetype defines what makes something a book. It does not gatekeep what others may say about it.

A chess game makes the pattern vivid. The game is an item. CHESS is its archetype, declaring the frames a chess game is expected to carry: the players, the moves, the result. The item itself, however, is not a monolithic structure. It is an accumulation of signed frames.

Players would register by signing their own PLAYER frames: PLAYER { (AGENT) = Fischer, (ROLE) = WHITE } signed by Fischer; PLAYER { (AGENT) = Spassky, (ROLE) = BLACK } signed by Spassky. Each player attests their own participation. It is not assigned by a third party; it is declared by the participant and carries their signature.

Then moves: MOVE { (LOCATION) = the-game, (AGENT) = Fischer, (THEME) = king-pawn, (SOURCE) = e2, (GOAL) = e4 } signed by Fischer. Each move is independently meaningful, independently signed, independently verifiable. The game is the ordered sequence of these signed assertions, all cohering around the same item identity.

Because each move is a self-contained signed frame, the game can be played peer-to-peer: each move travels directly from the player who made it to the other, with no referee server, no hosted backend, no central authority. The game is recorded by being signed; it does not need any third party to become real. And because each move is a frame, it is queryable: "all games where someone opened e4" is an index lookup on MOVE frames with (GOAL) = e4.

The pattern generalizes immediately. A chat room would be an item with signed MEMBERSHIP and MESSAGE frames. A key log would be KEY, REVOKE, and DELEGATE frames. An auction would be signed BID frames. All the same pattern: an item exists, people make signed assertions on it, and those assertions collectively define what it is.

The photograph, revisited

This paper opened with a photograph: a user saves it, and no layer of the stack knows what it is, who is in it, what occasion it documents, how it relates to other photographs or to the people depicted. In the picture being described here, a photograph is an item. Its archetype is PHOTOGRAPH. The information that today lives only in the user's head, or in a proprietary application's database, would be captured as frames at the moment of creation.

The photographer takes a picture of Alice and Bob at Alice's graduation. The item is created, and with it, the frames:

```

IMAGE      { (THEME) = the-photo, (VALUE, JPEG) = <image data>, (VALUE, JPEG, THUMBNAIL) = <
DEPICTS    { (THEME) = the-photo, (AGENT) = Alice, (LOCATION) = [120px, 340px, 280px, 510px]
DEPICTS    { (THEME) = the-photo, (AGENT) = Bob, (LOCATION) = [400px, 320px, 560px, 500px] }
OCCASION   { (THEME) = the-photo, (TOPIC) = graduation }
PLACE      { (THEME) = the-photo, (LOCATION) = [37.4275°N, 122.1697°W] }
CAPTURED   { (THEME) = the-photo, (INSTRUMENT) = iPhone, (TIME) = 2024-06-15 }

```

Each frame is a separate assertion. Each is signed by the photographer. Each is indexed by the meanings in its binding keys. The photograph does not merely *have* this information attached to it. It is this information, structured as grounded semantic assertions.

Now the queries that no existing layer can answer become index lookups:

- “All photographs of Alice” finds DEPICTS frames where (AGENT) = Alice.
- “All graduation photos” finds OCCASION frames where (TOPIC) = graduation.
- “Photos taken near this location” finds PLACE frames whose (LOCATION) coordinates fall within a radius.
- “Photos taken by this device in June 2024” filters CAPTURED frames by INSTRUMENT and TIME.

No crawling. No NLP. No reconstruction. The meaning was captured when the photograph was created, by the person who knew what the photograph was of.

The archetype PHOTOGRAPH declares that photographs are expected to carry DEPICTS, PLACE, and CAPTURED frames. The accumulation surface is open regardless. Later, someone else adds:

```

LIKE       { (THEME) = the-photo, (AGENT) = Carol, (VALUE) = "Great shot!" }
FUNNY      { (THEME) = the-photo, (AGENT) = Jeff, (VALUE) = "😄" }
RECOMMEND  { (THEME) = the-photo, (AGENT) = Dave, (RECIPIENT) = Eve }
DEPICTS    { (THEME) = the-photo, (TOPIC) = sunset }

```

Each is a first-class assertion with its own predicate and roles, not a flat comment or tag. The photograph accumulates reactions, recommendations, and additional descriptions the same way a chess game accumulates moves: signed assertions from identified parties, cohering around the same item identity. The PHOTOGRAPH archetype did not mention LIKE, FUNNY, RECOMMEND, or a third-party DEPICTS. It did not need to.

A Spanish speaker sees the same photograph through Spanish lexemes. The frames are the same; the words that surface them differ. No translation has occurred. The meanings were language-neutral from the start.

The architecture closes a circle: even sememes themselves would be items. The sememe METER carries a GLOSS frame in English, a GLOSS in Spanish, a DIMENSION frame (LENGTH), CONVERSION frames to other units, a HYPERNYM frame (METER is-a LENGTH_UNIT), and a SYMBOL frame (“m”). The meaning is not a definition string but the structured totality of everything asserted about it. A language itself (English, Spanish, Japanese) is an item whose frames include its entire lexicon. The vocabulary lives *in* the graph, as items made of frames, using the same primitives as everything else. Your local runtime carries the vocabulary you have encountered; when you meet a sememe you have not seen, you fetch it from peers who have it, the same way you fetch any other item.

This is where the analogy to files becomes concrete:

Files	Items
Opaque bytes; no layer interprets content	Typed frames; the layer knows what everything means
Named by path in a hierarchy	Discoverable by meaning; exist in a semantic graph, not a tree
No built-in authorship, versioning, or integrity	Every frame is signed, content-addressed, and verifiable

Files	Items
Metadata is a sidecar (EXIF, xattr, .DS_Store) “Relatedness” means same folder or a hyperlink	Metadata IS frames, first-class and queryable Typed, signed, indexed, traversable semantic links
Application decides how to interpret it	Item carries its own vocabulary and presentation
Search by filename or keyword	Query by meaning across the graph

The item is what would replace the file for the user. Not at the POSIX level (bytes and streams are a fine substrate for low-level I/O) but for user-facing data: the things people create, name, share, organize, search for, and care about. The item is the thing that knows what it means, because it is made of frames, and frames are meaning.

7. The Shared Meaning Space

A semantic frame is only as useful as the vocabulary it draws from. If every application defines its own predicates and roles, frames reproduce the same fragmentation as string-keyed pairs, just with more structure.

Previous sections have named the linguistic resources the vocabulary is anchored in: WordNet’s 120,000 synsets, CILI’s cross-lingual concept identifiers, FrameNet’s 1,200+ frame definitions, VerbNet’s verb-class-to-role mappings, ISO 24617-4’s standardized role inventory, and the three structural additions (VALUE, CONFIG, FOLLOWS) that adapt the linguistic inventory to the needs of a data primitive. Those resources are not repeated here. What this section addresses is how the vocabulary works as a *shared space*: how it handles specific entities, how it relates meaning to expression across languages, and how it grows.

The shared vocabulary solves a problem that prior efforts struggled with. Gruber (1993) argued that shared ontologies are essential for knowledge sharing; Lenat’s CYC (1995) tried to build one by hand-encoding millions of common-sense assertions, demonstrating both the importance and the intractability of that approach; the Semantic Web’s ontology languages (RDF, RDFS, OWL) produced a proliferation of competing ontologies that became a problem in itself. The anchor available now is different: not hand-authored axioms or ontology-language constructs, but the empirically documented structure of human lexical semantics, built and validated by the computational linguistics community over decades.

The entity problem

The AUTHORED example: predicate AUTHORED, (THEME) = The Hobbit, (AGENT) = Tolkien. AUTHORED is a shared meaning. PERSON is a shared meaning. BOOK is a shared meaning. What about Tolkien *himself*?

Today, Tolkien exists as a Wikipedia page, an Amazon author page, a Goodreads entry, a TMDb profile, a Library of Congress authority record, a Wikidata entry, and countless other disconnected representations. None is the canonical Tolkien that every system could use as the AGENT binding.

This is the hardest problem the shared meaning space must address. WordNet provides the *concept* PERSON, but not an identity for every specific person. Every previous attempt at scale entity identity has hit the same tension: centralized registries (Wikidata, Library of Congress) are fragile, political, and exclusionary. Fully decentralized naming is ambiguous.

The position taken here is that entities are items: collections of frames with cryptographic identities. Tolkien, in this picture, is not a string or a URL or a row in a registry. He is an item carrying frames

that assert his name, birth date, works, relationships. These frames are signed by the people and institutions that assert them.

Convergence could happen through the social graph. When Alice creates an AUTHORED frame binding a Tolkien entity as AGENT, she binds to a specific cryptographic identity from her trust network. If the Library of Congress publishes a SAME_AS frame linking their authority record to Alice's Tolkien entity, and Bob trusts both, Bob's system could resolve them as the same entity. No central registry, only an accumulation of signed assertions from trusted parties. Each user's runtime performs the resolution against its own trust graph, which means different users may legitimately see different resolutions depending on whom they trust.

I will not pretend this is a solved problem. It trades the problems of centralized identity (political control, single points of failure) for different problems (convergence latency, conflicting identities). I believe the trade-off is correct for a decentralized semantic layer, but the entity problem remains the area where the architecture is most genuinely unproven.

Meaning and expression

The architecture described here would separate *meaning* from *expression*: meanings are language-neutral; words belong to specific languages and point to meanings. To “translate” a concept from English to Spanish, look up the English word's meaning, then find the Spanish word for that meaning. Import English and Spanish WordNet (both linked via CILI), and you have a bidirectional dictionary covering 120,000 concepts. Not a feature. A structural consequence of separating meaning from expression.

An open commons

The shared meaning space is not a closed vocabulary. Domain-specific communities can extend it with their own concepts (medical terminology, legal concepts, engineering standards), connected to the base through the same hierarchical relationships. New languages connect by linking their words to existing meanings. The vocabulary grows from the edges, not from the center: extensible without fragmentation, because every extension is anchored in the shared backbone. Extensions propagate the same way every other item does, through signed, content-addressed replication between trusting peers. A medical community defines its vocabulary among its own peers without permission from a central authority; anyone outside who wants access fetches it through the same substrate. The commons has no curator, only authors and trust.

8. Computation as Frames

The claim that semantic frames constitute a genuine base layer (not merely a metadata system) requires demonstrating expressiveness in domains far removed from natural language. Mathematics is the strongest test case: the most formal, least ambiguous domain of structured knowledge. If thematic roles can describe mathematical operations, they are not linguistic conveniences. They are universal structuring principles.

The mapping turns out to be natural.

Arithmetic

$3 + 5 = 8$. The operation ADD is the predicate. The operands are not Agents (they don't initiate anything) or Patients (they don't change). One is the Theme (the entity being operated on) and the other is the Instrument (the means by which the operation is performed). Natural language reveals the asymmetry: we say “add 5 to 3,” not “add 3 and 5 symmetrically.”

ADD { (THEME) = 3, (INSTRUMENT) = 5 }

The frame is the input form: a predicate and its bindings, nothing more. Evaluating the frame produces a value, in this case 8. That value plays the role of Result in the cognitive structure (the thing that comes into existence through the operation), but it is not a binding on the input frame. It is what comes out the other end when the frame is run against an implementation of ADD' s contract. Where that implementation comes from is taken up at the end of this section.

Subtraction confirms the asymmetry: SUBTRACT { (THEME) = 10, (INSTRUMENT) = 3 } evaluates to 7. Theme (“the thing being acted on”) and Instrument (“by what means”) are exactly the semantic functions the input values serve. The roles were defined for natural language, but they describe the same cognitive structure.

Calculus

The definite integral of x^2 from 0 to 1:

INTEGRATE { (THEME) = x^2 , (SOURCE) = 0, (GOAL) = 1, (INSTRUMENT) = dx }

Source and Goal for the bounds of integration. These roles were defined for physical motion (“move from the house to the store”) but they map onto abstract endpoints with no strain, because the cognitive structure is the same: a starting point, an ending point, a traversal. Evaluating the frame produces 1/3, the Result.

Differentiation and limits follow the same pattern: DIFFERENTIATE { (THEME) = x^2 , (INSTRUMENT) = x } evaluates to $2x$; LIMIT { (THEME) = $1/x$, (GOAL) = ∞ } evaluates to 0, the variable approaching the Goal the same way a physical object approaches a destination.

The role mapping

Math concept	Thematic role	Linguistic parallel
Operand / expression being operated on	Theme	“the thing being acted on”
Second operand / applied quantity	Instrument	“by what means”
Lower bound / starting value	Source	“where from”
Upper bound / ending value	Goal	“where to”
Answer / output	Result	“what comes into existence”
Both sides of an equation	Pivot	“central participant in fixed state”
Degree or magnitude of change	Extent	“by how much”
Path of integration (line integrals)	Path	“the route taken”

(Result here names the role for the *output* of evaluation, not an input binding on the frame. The other rows describe input bindings.)

Why this matters

The mapping is significant not because it enables a math engine (though it does: 5 meters + 3 feet is an ADD frame whose operands are quantities with unit sememes, resolvable because METER and FOOT are both LENGTH units with known conversion factors). It is significant because it demonstrates that thematic roles are cognitive structuring principles, not linguistic artifacts.

Mathematics and natural language both express: what is being operated on (Theme), by what means (Instrument), where we start (Source), where we end (Goal), by how much (Extent), and what results (Result). The roles are the same because the underlying cognitive operations are the same.

If ~25 thematic roles can structure natural language, social interactions, and mathematical expressions, those roles are genuinely universal. A base layer built on them is as general as meaning itself.

Mathematics as a language

As noted in section 5, predicates carry parsing behavior: + is a token whose corresponding sememe ADD declares itself as infix with a precedence and associativity. The consequence for mathematics specifically is that natural language, mathematical expressions, and domain-specific notations coexist within a single input stream. “Create chess where score > sqrt(9) named rematch” mixes English, a mathematical sub-expression, and an auxiliary predicate. One resolution pipeline; the language being spoken is inferred from the tokens, not assumed.

Mathematical expressions are not bolted onto the side of a semantic layer. They are frames. A spreadsheet cell is a frame whose value is the result of an expression frame. The boundary between “data” and “computation” dissolves the same way “data” and “metadata” does: both are role bindings on predicates.

The contract and the code

A predicate carries a *contract*: it declares what values it expects, what result it produces, and how it might be parsed and evaluated. The contract, however, is not the code that satisfies it. ADD can declare that it takes two operands and produces a sum; it cannot, by itself, compute the sum.

The answer that fits the rest of the architecture is that code is published the same way every other thing is: as items. An implementation of ADD would itself be an item, carrying the executable form (source code, compiled bytes, or a formal specification) alongside frames declaring which contract it satisfies, who signed it, and what runtime is needed. The relationship between an implementation and the predicate it satisfies is itself a frame.

A predicate could have many implementations: different runtimes, different trade-offs, different authors, all coexisting. A runtime evaluating an ADD frame would pick an implementation it can execute and whose author it trusts. The contract is the meaning; the code is the machinery. Nobody would own the contract. ADD is a sememe in the shared vocabulary, no different from BOOK or AUTHORED. Anyone could publish an implementation item, sign it, and let it propagate; whether a particular runtime uses it would be a matter of local trust, not central authorization.

Code distribution would become a special case of data distribution. Today, software reaches users through centralized clearinghouses: app stores, package managers, vendor release servers. In the picture being described, code would travel the same peer-to-peer mechanism as any other item: signed, content-addressed, replicated through relationships, versioned, forkable. The package manager dissolves into the same medium that carries the rest of the data.

The choice raises serious security concerns. Running code from arbitrary peers is a recipe for disaster unless the runtimes loading and executing it are properly sandboxed. Sandboxing in this picture would not be a separate special system. It would be another kind of policy attached to frames, the same way replication or retention policy would be. The problem is hard, but it is the same kind of hard as sandboxing untrusted JavaScript in a web browser, and the existing landscape of techniques (capability-based interfaces, isolated execution environments, formal verification of restricted languages) gives plenty to draw on.

Because code is an item and data is an item, one of the main structural rationales for SaaS centralization dissolves. Most software became a service in part because the operator held both the code and

the data, and running the code required their infrastructure. Here, both travel through the same peer substrate, and execution happens wherever the user has a runtime, most naturally on their own device. The server farm, the subscription, the operator as gatekeeper: these arrangements lose the structural basis that made them seem like the only option.

The deeper point is structural. Computation would not need a separate apparatus alongside the data. The contract for a piece of computation is a meaning in the vocabulary, played as a predicate. The code that satisfies the contract is an item in the graph. The link between them is a frame. Everything that has to exist for computation to happen is the same kind of object that has to exist for everything else.

9. What Follows

If we accept the premises of both pillars, that meaning must live in the data and computation must live with the user, then a number of consequences would follow. They would not be independent features. They would be structural properties, coupled: you could not get some without the others, and you would not need to engineer them separately.

Queryability without crawling. Every piece of data would be a frame with a grounded predicate and semantically-keyed bindings. The data *would be* the index. “All books authored by Tolkien” would not be a text search; it would be a lookup on AUTHORED frames where AGENT refers to Tolkien. Each frame would be indexed by its predicate and by each meaning in its compound binding keys. For N frames with K bindings on average, the index contains $O(N \times K)$ entries. Queries resolve in $O(\log N)$. Standard data structures, richer keys.

Multilingual interoperability. A Spanish speaker and an English speaker would see the same data through their own words but operate on the same semantic structures. The layer would not translate; it would resolve, through different words, to the same concept.

Trust as data. Every frame would be a signed assertion by an identified party. A “like” would be a signed frame. A spam label would be a signed frame. A fact-check would be a signed frame. Different users, with different trust relationships, would see different views of the same underlying data, not because a platform is making editorial decisions, but because trust policies (themselves data) produce different evaluations. This is Szabo’s (1997) vision of formalizing relationships on public networks, realized through the frame primitive.

Content-addressed identity. Frame identity would be determined by semantic content (predicate + bindings). Two identical assertions produce the same identity regardless of who makes them or when. Content-addressing (Merkle, 1979; Benet, 2014) applied to semantic structures rather than opaque bytes.

Composability. A document is frames. A chat room is frames. A chess game is frames. A trust relationship is frames. A mathematical expression is frames. There is no structural distinction between content, metadata, relationships, configuration, and computation.

Liveness. Real-time shared presence would not be a separate system. A PRESENT frame asserts “I am in this space” ; an AVATAR_STATE frame with a LATEST retention policy carries position at 60Hz; stream bindings carry video and audio. Three temporal modes (durable, ephemeral, streaming), one frame model. This is how Croquet’s (Smith, Kay, Raab, & Reed, 2003) vision of shared, replicated environments could be realized without a single runtime.

Syntax as vocabulary. Predicates carry their own parsing behavior. Operators declare precedence, functions declare grouping, prepositions declare role assignment. One resolution pipeline. Natural language, mathematics, chess notation, and any future domain syntax all flow through the same mechanism. Parsing is resolution.

Self-describing data. A frame carries everything needed to interpret it: its predicate names the kind of assertion, its binding keys name what each value means. No external schema, no application-specific decoder ring.

Subsumption of platforms. A product listing would be frames (PRICE, CATEGORY, LOCATION, DESCRIPTION, OFFER). A community would be frames (MEMBERSHIP, MODERATION, TOPIC, ANNOUNCEMENT, QUESTION). A review would be frames (RATING, TOPIC, AGENT). A citation graph would be CITES frames. A social network would be frames (FOLLOW, LIKE, CRITIQUE, RECOMMEND, BLOCK). Each is currently a proprietary database on a proprietary platform. In the shared meaning space, all would be the same primitive.

Applications without platforms. An email client, a document editor, a social feed, a project tracker would be applications over frames. Any runtime that understands the shared vocabulary could serve as the application; competing clients would read the same data. No user would be locked in to an interface because their data is readable only by its author's binaries. The user picks the client; the data does not belong to the client.

Offline by default. A local runtime with local data is trivially offline-capable. When a network returns, changes propagate to peers. Offline is a consequence of where data and computation live, not a feature to be engineered.

Resilience to vendor disappearance. Items live on users' devices and on the peers who have replicated them. A company that shut down or was acquired would lose the ability to ship updates, but the data, the tools, and the peer network would remain. The "long now" property that Kleppmann et al. name in local-first software is a structural consequence, not an engineering feature.

10. Authorship, Not Ownership

A popular slogan in the decentralization and crypto communities is "own your data." The phrase evokes the right sentiment. The lopsided relationship between users and platforms is unjust. Something must change. Users deserve agency over their own contributions. The word "ownership" applied to data, however, promises more than any technical system can deliver, and it is worth stating plainly what this proposal does and does not accomplish.

Ownership, in every meaningful sense, implies control. A thing I own is a thing I can exclude others from, a thing I can hold or withhold, a thing that is mine and not yours. These properties hold for physical objects because matter occupies space and cannot be in two places at once. They hold for artificially scarce digital assets like cryptocurrency tokens because the network's rules enforce the scarcity. They do not hold for ordinary data, and they cannot. Once I have given you a copy, I have no technical means of revoking it. You can keep it, forward it, publish it, transform it, or forget it. My wishes have no technical weight. This is not a failure of the substrate proposed here, or of any honest substrate; it is a property of copyable information, and any system that preserves human agency must accept it.

Before going further, a distinction worth naming. What the substrate technically delivers is *attribution*: the provable linkage between a signed assertion and the key that signed it. *Authorship* is the broader human concept, the social claim that a particular person created something. Under the usual assumption that the signer is the creator, attribution stands in for authorship closely enough that I will use the words almost interchangeably below, reaching for "authorship" when the human framing matters and "attribution" when the technical mechanism does. The two diverge in edge cases (ghostwriting, delegated signatures, AI-generated content), but those edges do not change the core claim: what the substrate provides is a reliable, verifiable linkage from assertions to keys. Whether the signer is the author is a social-layer question no substrate can answer.

What this proposal can accomplish is closer to the *spirit* of what people mean when they say "own your data," without claiming the part that cannot be delivered. You hold your own keys, and nobody

can sign as you. You author your own assertions, and nobody can forge them. You keep local custody of your data, and no vendor can revoke your access to work you already have. You choose which peers you share with going forward, and you do so through deliberate trust relationships rather than by default exposure to whoever hosts your platform. These are real, technically enforceable properties. They are not ownership of data. They are ownership of your *participation* in a network: your keys, your authorship, your custody, your consent to new copies.

What the proposal cannot do, nor should any honest proposal claim to do, is compel other parties to honor your wishes about copies they already hold. If you share risqué photographs with a trusted partner and the relationship goes wrong, no substrate can un-share what is already shared. The partial solution is social, not technical. A trust paradigm is what you have when you can *choose* whom to share with, and when the act of sharing is deliberate rather than default. If someone violates that trust, you can stop trusting them, and the trust graph as a whole responds: others who trust you see your revised posture and may update their own relationships with the offender. This is how human communities have always handled the problem. The substrate does not replace that social mechanism. It returns computing to a state where social mechanisms can actually function, because sharing stops being a default and becomes a choice.

Legal frameworks, contracts, and commercial licenses compose naturally with all of this, and the primitives CG provides (signed assertions, key-based identity, content-addressed verification) make them easier to encode when they apply. Nothing here displaces them.

The word “ownership” invites confusion because it borrows from property law what the medium cannot enforce. A more honest vocabulary for this project is authorship, custody, and consent. What you can have, in this proposal, is exactly those three: provable authorship of what you assert, local custody of what you hold, and a meaningful say in the propagation of new copies you make. That is substantially more than platforms currently permit, and substantially less than the “ownership” rhetoric suggests. The difference between those two is where honest design lives.

11. Honest Reckoning

I am not the first to propose an ambitious rethinking of how computing handles information. The history of such proposals is largely a history of instructive failures, and I would be foolish to ignore it.

Xanadu (Nelson, 1974) envisioned a global, versioned, bidirectional-linking document system. It got content addressing, versioning, and bidirectional links right (concepts that took decades to resurface in Git and IPFS). It failed because it demanded solving everything simultaneously before shipping anything. Lesson: ship incremental function, not a complete vision.

CYC (Lenat, 1995) set out to encode all common-sense knowledge as logical assertions. It got the diagnosis right: computers need world knowledge, not just data. It stalled because hand-authoring millions of axioms does not scale. Lesson: anchor in existing resources and let meaning emerge from use.

Croquet (Smith, Kay, Raab, & Reed, 2003), Alan Kay’s vision of shared, replicated 3D environments, got replicated state and seamless collaboration right. It faded because it required a complete runtime (Squeak Smalltalk) and could not interoperate with existing software. Lesson: platforms that cannot meet users where they already are face adoption cliffs no technical elegance can overcome.

Plan 9 (Pike et al., 1995) pushed Unix’s “everything is a file” to its logical conclusion. Technically superior to Unix in almost every way, it failed to displace it because it required abandoning the entire Unix ecosystem. Lesson: even a cleaner design loses to an entrenched ecosystem unless it provides a bridge.

The Semantic Web (Berners-Lee et al., 2001) got the diagnosis exactly right: the web needs machine-readable semantics. It built a rigorous stack that works in specialized domains. It did not become

general-purpose because it was layered *on top of* the web rather than built into it. Lesson: a semantic layer that is optional will remain marginal.

Local-first software (Kleppmann et al., 2019) is the tradition directly upstream of this paper's second pillar. Its seven ideal properties describe applications that live on the user's device, work offline, sync peer-to-peer, and retain user ownership and control. The tradition is healthy in research and in niche commercial software but has not displaced the SaaS default, because solving sync and interoperability without a central coordinator, while preserving user experience, has been hard enough that most teams took the easier centralized path. Lesson: the technical problems are now serviceable with current tools; what is missing is a substrate that makes local-first the easier path, not just a more virtuous one.

What do these teach?

Incremental delivery is non-negotiable. A system that requires completeness before it provides value will never reach completeness. Each increment must be useful on its own.

Build on existing resources. CYC tried to encode all knowledge manually. The Semantic Web required ontology engineering for every domain. I would rather anchor in WordNet, CILI, VerbNet, and ISO 24617-4: resources built and validated over decades by the computational linguistics community. I am not inventing a vocabulary; I am giving an existing, empirically validated vocabulary a new job.

Provide a bridge. Plan 9 and Croquet demanded that users abandon their ecosystems. A semantic base layer must coexist with files, filesystems, and the web. POSIX is a reasonable base layer for byte handling. It was never intended to be a base layer for meaning.

Neither pillar can be optional. This is the deepest lesson from the Semantic Web on one side and from local-first retrofits on the other. If creating semantic structure is a separate step from creating data, most people skip it. If local control is a separate feature to opt into, most people stay with the hosted default. The design must make creating data *be* creating semantic, user-held structure, the way writing a sentence is expressing meaning, not writing sounds and then separately annotating what they mean.

Can this proposal avoid the fates of its predecessors? Honestly: I do not know. The ambition is large, the history is cautionary, and the engineering challenges are real. The linguistic resources now exist, however. WordNet has 120,000 synsets. CILI links them across languages. VerbNet classifies 300 verb classes with role declarations. ISO 24617-4 standardizes the role inventory. UniMorph provides morphological data for 100+ languages. These resources represent decades of cumulative scholarly work. They did not exist when CYC began, when the Semantic Web was proposed, or when Croquet was built.

Neither did the technical infrastructure for the locality pillar. Modern signing cryptography (ed25519 and related primitives) is cheap enough to apply at the per-message level. Content-addressing is ubiquitous (every Git commit is a use of it). CRDTs have moved from research to shipping practice. Open-source P2P transport stacks (libp2p, modern QUIC implementations) are mature. A local-first substrate in 2026 is not conjuring machinery from nothing; it is composing pieces that have all shipped, some many times over.

And, worth stating plainly: AI assistance has compressed what was previously decades of solo implementation work into feasible timescales, and sustained dialogue with it has contributed to the clarity of the model as a whole. The bottleneck for ambitious software projects has always been the sheer volume of code required. That bottleneck has narrowed dramatically. This does not guarantee success, but it changes the economics of ambition.

The path forward is incremental: frames as a local data format; a shared vocabulary seeded from WordNet and CILI; a local runtime that stores, queries, and resolves data by meaning; and a peer-to-peer network where that data is exchanged between nodes connected by trust. Each step independently useful. Together, the semantic and local-first base layer that computing has been missing since the networked era began.

Whether it works is an empirical question. I offer it not as a certainty but as a proposal, grounded in established theory and validated resources, that the time is right to try.

References

- Baker, C. F., Fillmore, C. J., & Lowe, J. B. (1998). The Berkeley FrameNet Project. In *Proceedings of ACL/COLING*, 86-90.
- Barnes, J. A. (1954). Class and Committees in a Norwegian Island Parish. *Human Relations*, 7(1), 39-58.
- Benet, J. (2014). IPFS — Content Addressed, Versioned, P2P File System. arXiv:1407.3561.
- Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. *Scientific American*, May 2001.
- Bizer, C., Heath, T., & Berners-Lee, T. (2009). Linked Data — The Story So Far. *International Journal on Semantic Web and Information Systems*, 5(3).
- Bond, F., Vossen, P., McCrae, J., & Fellbaum, C. (2016). CILI: the Collaborative Interlingual Index. In *Proceedings of the 8th Global WordNet Conference*, 50-57.
- Bond, F. & Foster, R. (2013). Linking and Extending an Open Multilingual Wordnet. In *Proceedings of ACL*, 1352-1362.
- Bonial, C., Stowe, K., & Palmer, M. (2011). Renewing and Revising SemLink. In *Proceedings of the 2nd Workshop on Linked Data in Linguistics*.
- Bush, V. (1945). As We May Think. *The Atlantic Monthly*, July 1945.
- Clarke, I., Sandberg, O., Wiley, B., & Hong, T. W. (2001). Freenet: A Distributed Anonymous Information Storage and Retrieval System. In *Designing Privacy Enhancing Technologies*, Springer, 46-66.
- Cohen, B. (2003). Incentives Build Robustness in BitTorrent. In *Workshop on Economics of Peer-to-Peer Systems*.
- Engelbart, D. C. (1962). Augmenting Human Intellect: A Conceptual Framework. SRI Summary Report AFOSR-3223.
- Fillmore, C. J. (1968). The Case for Case. In Bach, E. & Harms, R. T. (Eds.), *Universals in Linguistic Theory*, 1-88. Holt, Rinehart & Winston.
- Fillmore, C. J. (1982). Frame Semantics. In *Linguistics in the Morning Calm*, 111-137. Seoul: Hanshin Publishing.
- Gruber, T. R. (1993). Toward Principles for the Design of Ontologies Used for Knowledge Sharing. Technical Report KSL 93-04, Stanford University. *Knowledge Acquisition*, 5(2), 199-220.
- Hewitt, C., Bishop, P., & Steiger, R. (1973). A Universal Modular ACTOR Formalism for Artificial Intelligence. In *IJCAI' 73*, 235-245.
- Hogan, A. et al. (2021). Knowledge Graphs. *ACM Computing Surveys*, 54(4).
- Kay, A. C. (1993). The Early History of Smalltalk. In *HOPL-II: History of Programming Languages*. ACM.
- Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-First Software: You Own Your Data, in spite of the Cloud. In *Onward! 2019*, ACM.
- Lenat, D. B. (1995). CYC: A Large-Scale Investment in Knowledge Infrastructure. *Communications of the ACM*, 38(11), 33-38.
- Merkle, R. C. (1979). *Secrecy, Authentication, and Public Key Systems*. Ph.D. dissertation, Stanford University.

- Milgram, S. (1967). The Small-World Problem. *Psychology Today*, 1(1), 61-67.
- Miller, G. A. et al. (1993). Introduction to WordNet: An On-line Lexical Database. Princeton University.
- Montague, R. (1973). The Proper Treatment of Quantification in Ordinary English. In *Approaches to Natural Language*, 221-242. Reidel.
- Nelson, T. (1974). *Computer Lib / Dream Machines*. Self-published.
- Palmer, M., Gildea, D., & Kingsbury, P. (2005). The Proposition Bank: An Annotated Corpus of Semantic Roles. *Computational Linguistics*, 31(1), 71-106.
- Pike, R. et al. (1995). Plan 9 from Bell Labs. *Computing Systems*, 8(3), 221-254.
- Ruppenhofer, J. et al. (2006). FrameNet II: Extended Theory and Practice. ICSI Berkeley.
- Shapiro, M. et al. (2011). Conflict-free Replicated Data Types. In *SSS 2011*, Springer.
- Smith, D. A., Kay, A., Raab, A., & Reed, D. P. (2003). Croquet — A Collaboration System Architecture. In *Proceedings of C5*, IEEE.
- Szabo, N. (1997). Formalizing and Securing Relationships on Public Networks. *First Monday*, 2(9).
- Tarr, D., Lavoie, E., Meyer, A., & Tschudin, C. (2019). Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications. In *ACM ICN '19*.
- Vossen, P. (1998). EuroWordNet: A Multilingual Database with Lexical Semantic Networks. *Computational Linguistics*, 25(4).
- Watts, D. J., & Strogatz, S. H. (1998). Collective Dynamics of 'Small-World' Networks. *Nature*, 393(6684), 440-442.
- Youn, H. et al. (2016). On the Universal Structure of Human Lexical Semantics. *PNAS*, 113(7), 1766-1771.